SBA301 / FINAL PROJECT DEFENSE

Nexora

Campus Incident Reporting and Maintenance Management

CampusFix project · SE1910 · Group 2
Lecturer: PhucPT

Team members & responsibilities

Phan Tuấn Thành / Team Leader 429218b · 13e9c00	Ticket workflow, review, assignment, reassignment, close
Tô Phi Oanh 86ccdab · d3dc91a	Catalog / asset UI, technician work, notifications
Trần Phan Nhật Huy 00d6186 · a7fe09e	Authentication, users, technician configuration, API regression
Trần Quốc Thái cf08221 · d4da212	Attachment access, dashboard, ticket reports and CSV
Lâm Thoại Thanh 4fb7f65 · e572384	Categories, dynamic forms, submission, keyword search

Ownership follows report §1.2 and inspected commits. Shared modules have contributions from several members.

The campus maintenance problem

An incident needs a clear owner and a visible path to completion

01	**Fragmented reporting**	Messages can omit the location, asset or evidence needed for action.
02	**Unclear responsibility**	Managers need to match a ticket with an eligible technician.
03	**Difficult follow-up**	Reporters need progress, repair evidence and a way to confirm completion.

Case-study motivation. The repository contains no measured campus survey or operational impact study.

Proposed solution & objectives



Structured reporting

Published category forms, location and eligible asset selection.

Accountable execution

Review, skill / coverage checks, assignment and technician work logs.

Traceable completion

Private images, status history, resolution summary and reporter close.

Current scope also includes role-aware dashboards, period reports / CSV and recipient-scoped notifications.

User roles & key features

R	**Requester**	Create and track own tickets. Close own RESOLVED ticket.
T	**Technician**	Read assigned history. Start, log, upload and resolve as current assignee.
M	**Manager**	Review, evaluate eligibility, assign / reassign, view reports and assets.
A	**Admin**	Manage users, roles, catalog and published forms. Also manage tickets.

Management roles alone cannot close for a reporter or perform technician work. Backend checks role, relation and state.

Technology stack

React 19.3.0

Vite 8.3.0 · React Router 8.4.0
TanStack Query 5 · Axios
React Hook Form / Zod · Bootstrap

Java 21 / Spring Boot 3.5.16

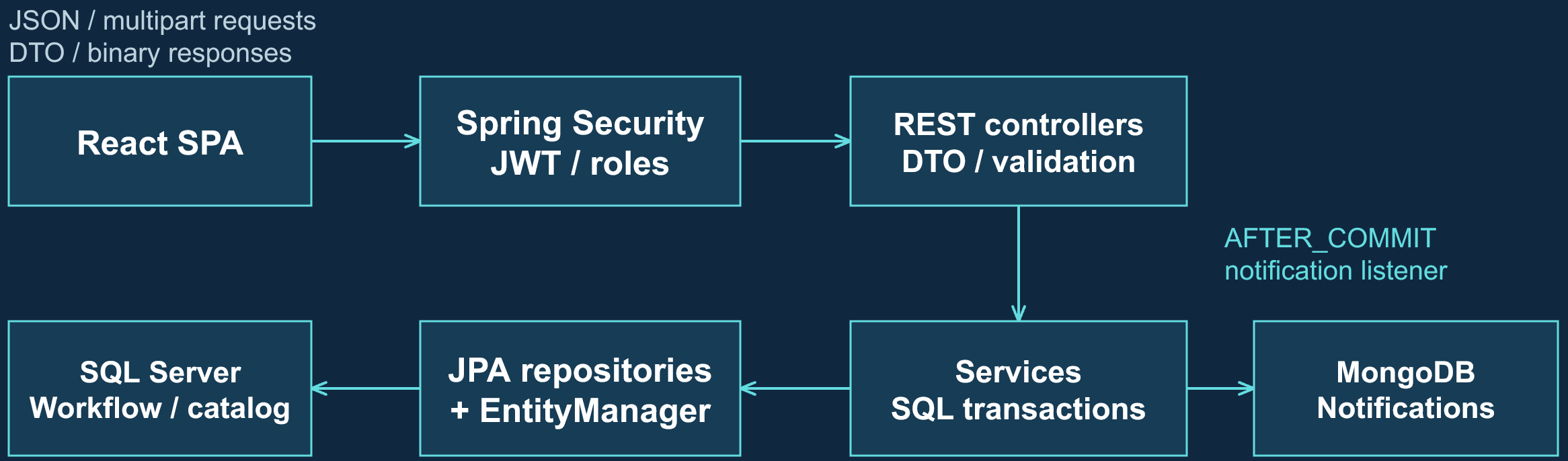
Spring Security · JWT / BCrypt
JPA / Hibernate · Flyway
SQL Server 2022 · MongoDB 8.0

Why these choices

Query owns server state. SQL owns relational workflow. MongoDB stores notifications and supporting documents.

Docker Compose runs the databases. JUnit 5 / Mockito / MockMvc / Testcontainers and Vitest / Testing Library support tests.

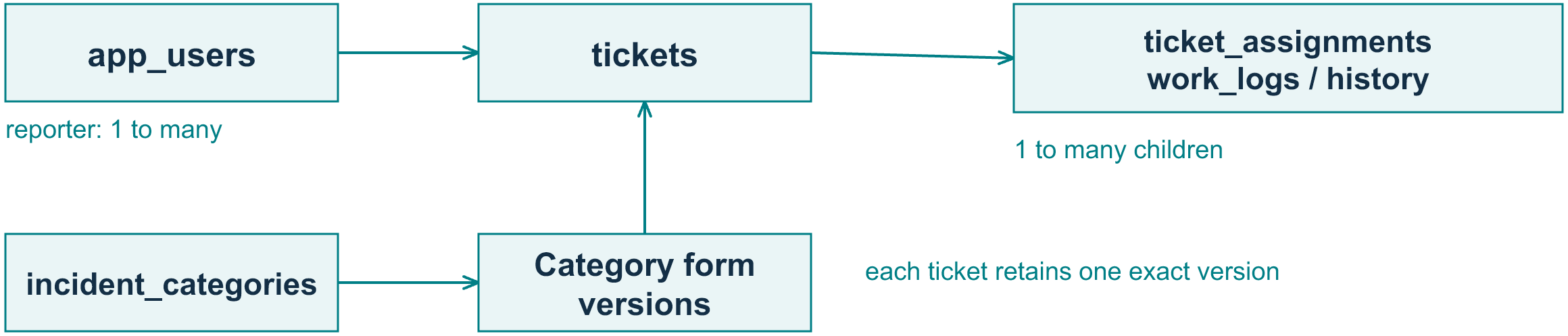
System architecture



Private filesystem: image bytes, with authorized content API access

One Spring Boot application: modular monolith. SQL, MongoDB and file storage have no shared distributed transaction.

Database design



Many-to-many

Users / roles via user_roles
Technicians / skills via technician_skills

SQL: workflow, forms, catalog and image metadata
MongoDB: notifications, audit and comment foundations

Simplified ERD. locations has many assets and tickets. MongoDB SQL IDs are logical references, without cross-store FKs.

Backend & REST API design

Representative endpoints / base /api/v1

POST /tickets	Multipart submission + typed dynamic answers
POST /tickets/{id}/assign	Version + technicianId, eligibility and state checks
POST /tickets/{id}/resolve	Version + outcome + summary, saved work log required
GET /reports/tickets/export	Manager / Admin period report as CSV



DTO + @Valid, domain validation and GlobalExceptionHandler: 400 input, 401 auth, 403 access, safe 404, 409 stale, 422 rule.

Authentication & authorization



01 **Bearer API requests**

Spring Security validates the signed token. Token lifetime is 8 hours.

02 **Role + object checks**

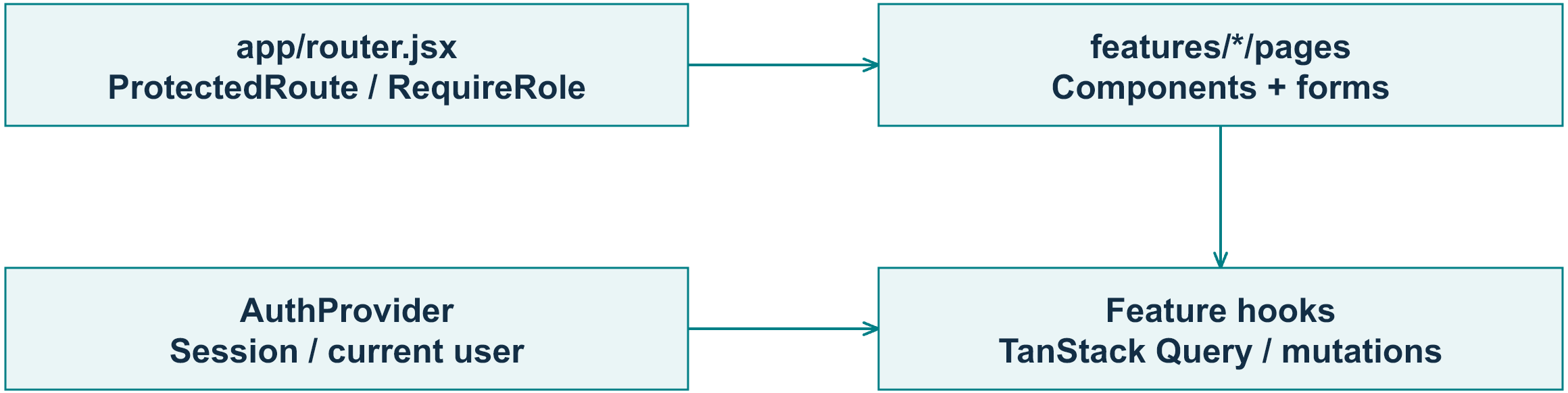
Ticket services reload the current user and enforce ownership or assignment.

03 **Protected SPA routes**

AuthProvider restores /auth/me. Axios adds the token from sessionStorage.

Logout clears client token and private Query cache. No server-side token revocation or refresh-token flow is implemented.

Frontend architecture

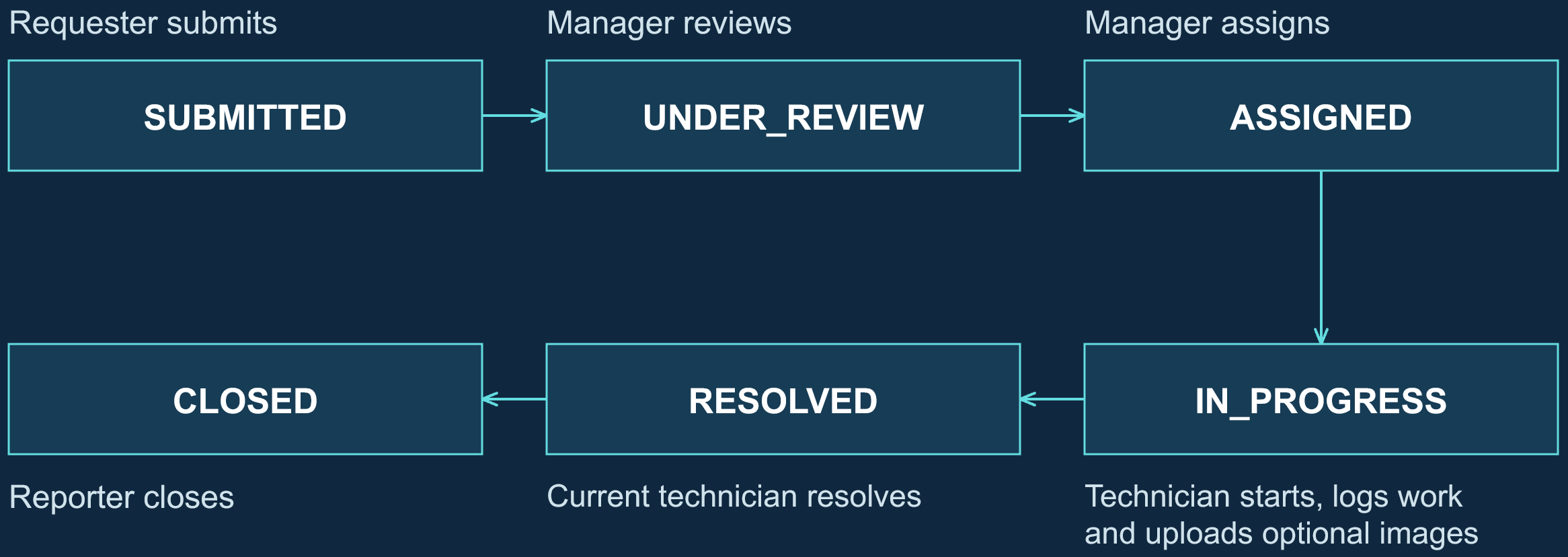


React Hook Form / Zod validate forms.
Page state covers loading, empty, error and success.

Feature API modules call shared Axios.
Mutation success invalidates relevant queries.

app / features / shared structure. Vite proxies /api to localhost:8080. Disabled mock flags keep the live demo on real APIs.

Core business workflow



React action → REST service → locked SQL update + history → DTO → Query refetch. Resolve requires a work log and summary.

Technical decisions & challenges

01 **Concurrent assignment**

Expected version + SQL locks recheck state and technician capacity before commit.

02 **Changing category forms**

Tickets retain the published form version. Draft / publish controls preserve historical meaning.

03 **SQL + files + MongoDB**

Rollback callbacks clean files. After-commit listeners retry notifications and deduplicate event keys.

Implemented safeguards have tests. Cross-store crash recovery remains limited: no durable outbox or shared transaction.

Testing & quality assurance

977 passed / 155 skipped

Maven verify · 0 failures / errors · 9 November 2026

Frontend	648 tests passed across 81 files. Lint and build passed.
Real database checks	239 passed / 2 skipped across 5 selected SQL Server / MongoDB suites.
Meaningful cases	Wrong role, stale version, capacity, rollback, typed answers and reporter close

JUnit 5 / Mockito / MockMvc / H2 + Testcontainers. Vitest / Testing Library / MSW. No coverage percentage is available.

Git & teamwork evidence



Auditable implementation

429218b Review / assignment
4fb7f65 Ticket creation
d4da212 Period report / CSV
a7fe09e API regression scripts

Integration evidence

Merge commits reference PRs #98, #97 and #96.
GitHub Actions defines backend and frontend checks.

Local Git proves branch / commit / merge history. Live PR review approvals, issue-board status and weekly minutes remain unverified.

Live system demonstration

00:00–01:20	**Thoại Thanh**	Requester submits a ticket and refreshes
01:20–02:30	**Tuấn Thành**	Manager reviews, checks eligibility and assigns
02:30–04:15	**Phi Oanh**	Technician starts, logs work, uploads and resolves
04:15–05:10	**Nhật Huy / Thành**	Reporter checks the result, closes and refreshes
05:10–06:00	**Quốc Thái**	Manager report / CSV and recipient notification

Run the live SPA + API + SQL Server + MongoDB. Use separate browser profiles for Requester, Manager and Technician. Passwords stay off-screen.

Conclusion & future work

A complete core ticket journey

React and Spring Boot integrate structured reporting, controlled assignment, work evidence and reporter confirmation.

Current limits

Unassign has no runtime API. Reject / cancel / reopen, comments and feedback remain disabled.

Next priorities

Resolve CRUD / Delete requirement gaps, add coverage reporting, and improve file / notification recovery.

Thank you / Questions